Reviewer Pack — Minimal Rank of Hodge Decomposition in Algebraic Geometry vs…

Entry 543e00bf1cb8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 00:43:04 UTC

Minimal Rank of Hodge Decomposition in Algebraic Geometry vs ACC⁰ Circuit Size

Entry ID: `543e00bf1cb8` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 00:43:04 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Decomposition) **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity

Statement:

> ['For any explicit function f in P computed by an ACC⁰ circuit C with size S(n), the Hodge decomposition of the algebraic variety V defined by C yields a Hodge space of rank R(n) ≤ Ω(S(n)/log n).', 'Furthermore, there exists an algorithm to compute R(n) for instances of size n ≤ 40 in time O(T(n)), where T(n) = S(n)^2 + n log^3 n.', "For any explicit function f in P computed by an ACC⁰ circuit C with size S(n), if the Hodge decomposition of V has rank R(n) < Ω(S(n)/log n), then there exists a smaller ACC⁰ circuit C' computing f."]

Rationale (proposer's reasoning):

> ['Hodge Decomposition provides a systematic way to analyze the structure of algebraic varieties, which may reveal hidden complexities in the circuits computing explicit functions.', 'The connection between Hodge decomposition and ACC⁰ circuit size could potentially lead to new techniques for proving lower bounds on ACC⁰ circuits, as Hodge spaces have been used to study other complexity-theoretic objects.', 'This conjecture suggests a novel way of using algebraic geometry to understand the complexity of computing explicit functions in P.']

Taxonomy category: `ACC_SIPSER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7ae7784b390820c2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all ACC⁰ circuits C with size S(n) ≤ 40, the Hodge rank R(n) satisfies R(n) ≤ Ω(S(n)/log n), and falsified if there exists a circuit C with R(n) > Ω(S(n)/log n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge decomposition" AND "ACC0 circuit complexity" - "algebraic geometry" AND "circuit size analysis" - "minimal rank Hodge space" IN TITLE

Top relevant hits considered: - [http://arxiv.org/abs/1810.12657v1] The effect of a country's name in the title of a publication on its visibility and citability - [http://arxiv.org/abs/cs/0207003v1] Analysis of Titles and Readers For Title Generation Centered on the Readers - [http://arxiv.org/abs/1004.2719v1] Is This a Good Title?

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_acc0_circuit(n):
        # Simplified generation for demonstration purposes
        return [random.randint(1, 2) for _ in range(n)]

    def hodge_rank(C):
        S_n = sum(C)
        n = len(C)
        if n == 0:
            return Fraction(0)
        return Fraction(S_n, math.log(n + 1))

    def is_valid_circuit(C):
        # Simplified validation for demonstration purposes
        return all(x in [0, 1] for x in C)

    instances_tested = 0
    conjecture_holds = True
    counterexample = ""
```

```
    for _ in range(30):  # Ensure at least 30 instances per seed
        n = random.choice([5, 10, 15, 20, 30, 40])
        C = generate_acc0_circuit(n)

        if not is_valid_circuit(C):
            counterexample = "Invalid circuit generated"
            conjecture_holds = False
            break

        R_n = hodge_rank(C)
        instances_tested += 1

        if R_n > Fraction(S_n, math.log(n + 1)):
            counterexample = f"Counterexample found: R({n}) = {R_n}, expected ≤ Ω(S({n})/log n)"
            conjecture_holds = False
            break

    return {
        "metric_name": "Hodge Rank",
        "metric_value": hodge_rank(generate_acc0_circuit(40)),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b5cb733c.py", line 73, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b5cb733c.py", line 59, in run_trial
    "metric_value": hodge_rank(generate_acc0_circuit(40)),
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b5cb733c.py", line 30, in hodge_rank
    return Fraction(S_n, math.log(n + 1))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12236
2	preregistration	ollama_remote	glm4:latest	0	0	5978
3	novelty	ollama_remote	glm4:latest	0	0	4539
4	novelty	ollama_remote	glm4:latest	0	0	6134
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14656
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8183
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7492
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8458
9	judge	ollama_remote	glm4:latest	0	0	8183

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 75859 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/543e00bf1cb8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/543e00bf1cb8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/543e00bf1cb8.tar.gz` (if generated)